Clean Code

CHAPTER 1&2 –CLEAN CODE ROBERT C MARTIN

Clean Code

- We all know code represents the details of requirements.
- We know good code matters because we've had to deal with it for so long.

Emphasize on Writing Good Code

- I know of one company that, in the late 80s, wrote a killer app. It was very popular, and lots of professionals bought and used it.
- But then the release cycles began to stretch. Bugs were not repaired from one release to the next.
- Load times grew and crashes increased. I remember the day I shut the product down in frustration and never used it again.
- The company went out of business a short time after that.

Emphasize on Writing Good Code

- Two decades later I met one of the early employees of that company and asked him what had happened. The answer confirmed my fears. They had **rushed the product to market and had made a huge mess in the code**.
- As **they added more and more features, the code got worse and worse** until they simply could not manage it any longer. It was the bad code that brought the company down.

The Art of Clean Code?

- Let’s say you believe that messy code is a significant obstacle.
- Let’s say that you accept that the only way to go fast is to keep your code clean.
- Then you must ask yourself: “How do I write clean code?”
- It’s no good trying to write clean code if you don’t know what it means for code to be clean!
- The bad news is that writing clean code is a lot like painting a picture.
- Most of us know when a picture is painted well or badly. But being able to recognize good art from bad does not mean that we know how to paint. So too being able to recognize clean code from dirty code does not mean that we know how to write clean code!

What is Clean Code??

- *Clean code is code that is **easy to understand and easy to change**.*
- *Clean code is **simple and direct**. Clean code reads like well-written prose.*
- *Clean code follows the designer's intent and is full of crisp abstractions and straightforward lines of control.*

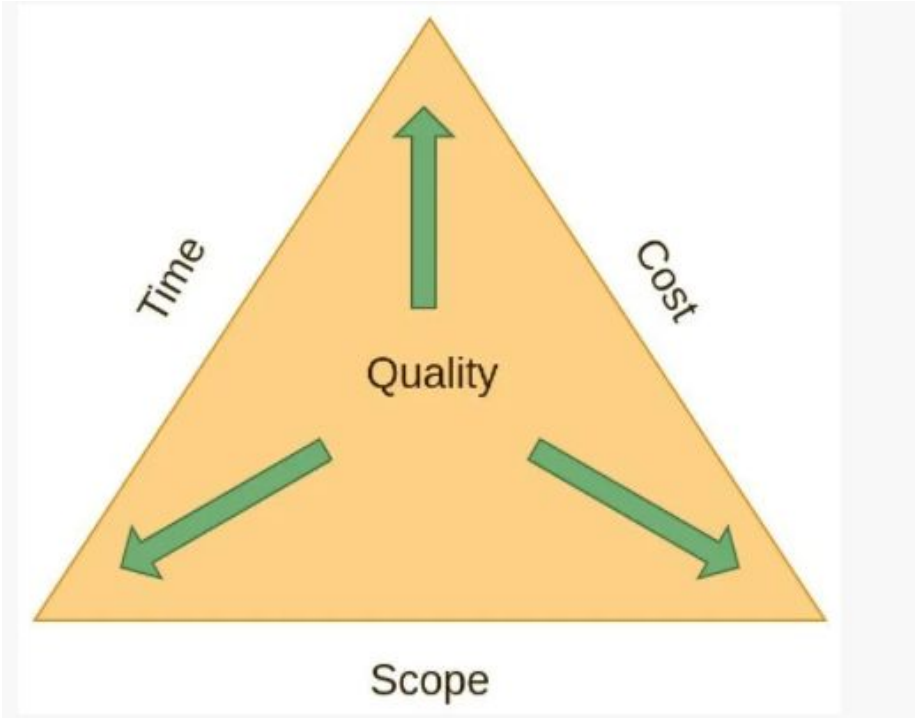
Beck Rules of Simple Code

- *Runs all the tests;*
- *Contains no duplication;*
- *Expresses all the design ideas that are in the system;*
- *Minimizes the number of entities such as classes, methods, functions, and the like.*

Who Cares About Clean Code ?

- One of the biggest issues within the software development industry is that, not many of the right people actually care about clean code.
- It’s true that maybe a certain percentage of developers may care about clean code, but they don’t all work on the same teams, and most probably not on your team.
- The truth is, *clean code takes time, effort, attention and care,* all four values the business sector doesn’t really care about.
- The reason being is that Clean Code falls into the *Quality criteria* of the Quality Triangle, a variant of what is most commonly known as the *Project Management Triangle*.

Clean code impacts quality



Clean code impacts quality

- So you can deliver code and deploy applications fast to meet business objectives, or you can take more time and ensure the quality of the code released.
- The problem is you can never have both, at some point something has to be dropped. More often than not, in software it's the quality.
- The primary reason why, is that code is often the least visible aspect of software development. Sure developers care about it, you'll often hear developers talk of *"beautiful code"* or even *"elegant code"* , but very rarely
- The only thing, users generally care about with software applications is that it solves a problem they have . You don't need clean , beautiful or even elegant code to solve problems, this can be done with just code.

How To Write Clean Code

Meaningful Name-Introduction

- Names are everywhere in software.
- We name our variables, functions, classes, and packages.
- We name our source files and the directories that contain them. We name our jar files and war files.
- We name and name and name. Because we do so much of it, we'd better do it well. What follows are some simple rules for creating good names.

Rules For Creating Good Names

- **Use Intention-Revealing Names**
  - It is easy to say that names should reveal intent.
  - The name of a variable, function, or class, should answer all the big questions.
  - It should tell you why it exists, what it does, and how it is used. If a name requires a comment, then the name does not reveal its intent.

Use Intention-Revealing Names

```
int d; // elapsed time in days
```

The name `d` reveals nothing. It does not evoke a sense of elapsed time, nor of days. We should choose a name that specifies what is being measured and the unit of that measurement:

```
int elapsedTimeInDays;
int daysSinceCreation;
int daysSinceModification;
int fileAgeInDays;
```

Avoid Disinformation and Encodings

Don't refer to a group of accounts as "accountList," whereas it is actually not a List. Instead, name it "accountGroup," "accounts," or "bunchOfAccounts." Avoid unnecessary encodings of datatypes along with the variable name.

```
var accountGroup: String?
var bunchOfAccounts: String
```

It may not be necessary where it is very well known to the entire world that in an employee context, the name is going to have a sequence of characters. The same goes for Salary, which is decimal/float.

Make Meaningful Distinctions

If there are two different things in the same scope, you might be tempted to change one name in an arbitrary way.
What is stored in emp, which is different from employee or employer ?

```
var emp: String?
var employee: String?
var employer: String?
```

How are these classes different?

```
class ProductInfo
class ProductData
```

How do you differentiate between these methods in the same class?

```
func getActiveAccount()
func getActiveAccounts()
func getActiveAccountsInfo()
```

So, the suggestion here is to make meaningful distinctions.

Use Pronounceable Names

Ensure that names are pronounceable. Nobody wants a tongue twister.
This

```
var absdegpoi123: Date?
var  modymdhms: AccountGroup?
Int pszqint;
```

vs

```
var generationTimeStamp: Date?
var employeeAccount: AccountGroup?
var employeeAccessCount: Int?
```

Don’t Be Cute/Don’t Use Offensive Words

- Don’t use funny names or any sort of slang while naming your items.

- Again , focus on the clarity. Say what you mean , Mean what you say.

Don’t Be Cute/Don’t Use Offensive Words

- Example

```
deleteAllItems()
```

vs

```
whack()
```

vs

```
kill()
```

Avoid Mental Mapping

- Let”s start with an example

```
r
t
p
```

- Even if we are developers, we don”t want to learn and memorize that “r” means full url of NUML server from Pakistan, “t” is the same url but without protocol information and “p” represent the query parameters.
- Use the following
  - urlNUMLPakwithPrtocol
  - urlNUMLPak
  - qryParameters

CamelCasing

- Camel case is the practice of capitalizing the first letter of each word when writing compound words or phrases, with the exception of the first word. Examples include:

– eBay

– eBook

– iPhone

– macOS

– vCard

CamelCasing

- If you also want to capitalize the first word, then that is called upper camel case, or pascal case. For instance:

– BlackBerry

– ContentEd

– LinkedIn

– PlayStation

– YouTube

CamelCasing

- CamelCasing Improves Readability
  - Camel case improves understanding and readability for everyone. It offers clear, scannable information to our audiences to make their lives easier.
  - This helps:

– People in a hurry

– People with low literacy

– People who are stressed

– People who are multi-tasking

– People with low English fluency

snake_case

- Snake case (stylized as snake_case) refers to the style of writing in which each space is replaced by an underscore (_) character, and the first letter of each word written in lowercase.

snake_case

- Here are three simple examples of code in the snake case naming convention:

  - INTEREST_RATE
  - increase_count_by_one
  - FIND_ALL_USERS

Pick One Word per Concept

- If all three methods do the same thing, don’t mix and match across the code base. Instead, stick to one.

```
dataFetcher() vs dataGetter() vs dataFetcher()
```

Pick One Word per Concept

- Use the same concept across the codebase.

- If you are using FetchValue() to return a value of something, use the same concept; instead of using FetchValue() in all the code, using getValue(), retrieveValue() will confuse the reader.

```
FetchValue() vs GetValue() vs RetrieveValue()
```

Don't Pun

This is exactly opposite of previous rule.
Avoid using the same word for two different purposes.
Using the same term for two different ideas is essentially pun.
For example, say you have two classes having the same add() method.
In one class, it performs addition of two values, in another, it inserts an element into a list.
So, choose wisely; in the second class use insert() or append() instead of add().